

项目通俗导览:这个项目到底在干什么(零基础版,课 0)

写给你自己看的"全项目说明书":不看代码、不懂术语,读完这份就能讲清我们在做什么、

为什么这么做、比赛怎么赢。文中数字都带口径标签,数据章数字=官方口径(0706 切换),

§7 故事章保留当时的历史口径快照(括号注明);最新值以 `python sim/headline.py` 输出为准。

看不懂的词查文末词汇表。

0. 三十秒版本

比赛题目:给一面 20 列 $\times$ 20 层的立体货架(400 个格子)设计一套"聪明的放货规则",

让仓库**取货快、重货稳、耗电少**,并且把这套规则装进一台工业电脑(ABB 的 PLC)真正跑起来。

我们的答案:**两层算法**——平时来一箱放一箱用"打分器"(快、简单、可解释),

攒够一批货用"整理师"(全局优化,给出理论上界)。这套东西已经全部做完并互相验证,

剩下 40 天的仗是把它**讲清楚**(报告/PPT/录屏),8 月 15 日提交。

1. 这道题在问什么(先把题目翻译成成人话)

想象一面巨大的快递柜墙:20 列宽、20 层高,取送货口在**左下角**。

一台"堆垛机"(可以理解成一台会横着跑、同时竖着升的取货电梯)负责存取。

题目给的现实约束:

- **越高的格子越脆弱**:最底层能放 100kg,每往上一层承重减 2%(第 19 层只能放 62kg);
- **有些格子已被占用**: $x+y$ 之和能被 3 整除的格子预设有货(133 个,呈对角条纹;官方 0706 答复口径,含 (0,0) 格);

- 每箱货有五个属性:编号、名称、**重量**、**访问频次**、**体积**。

要优化的三件事互相打架:

- **取货快**:常用的货放门口 → 可是门口格子就那么几个;
- **重货稳**:重的放低层 → 可是低层也是离门口最近的黄金位,放了重货,常用货就挤不进;
- **省电**:货举得越高越费电 → 又是一票"往低放"的需求来抢低层。

所以这不是"排个序"的小事,是一道**多目标冲突**的分配问题。学术界有个名字叫

SLAP(仓位分配问题),研究了六十年。

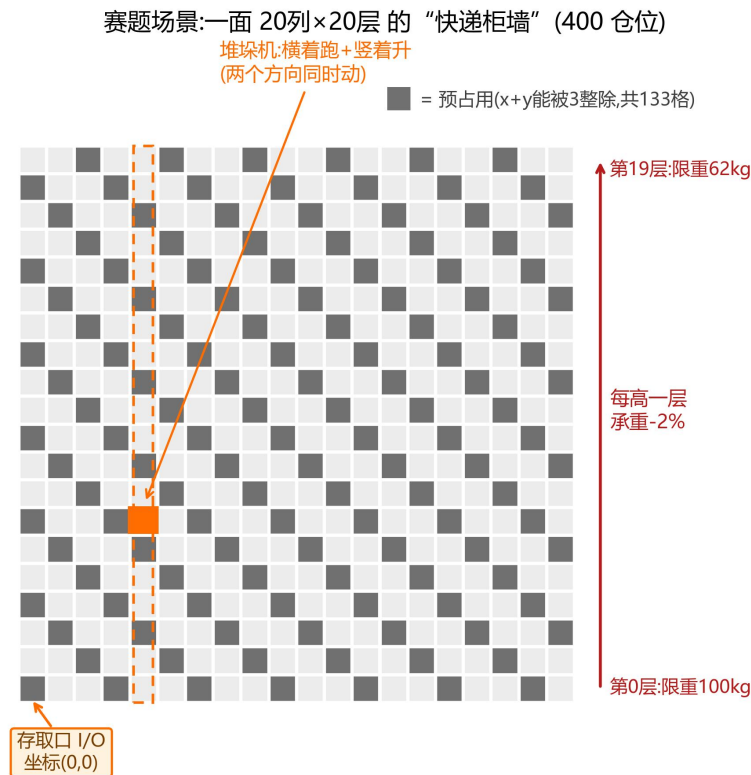


图 E1 赛题场景:把题面变成一张图(示意图,生成:python sim/explainer_figs.py)

2. 我们现在用的是什么算法(两层,各管一件事)

第一层:在线评分(平时值班的"打分器")

货是一箱一箱到的,来了就得马上决定放哪,不能等。我们的做法:每来一箱货,

给 400 个格子**每个都打一个分**,分数最低的格子胜出。打分公式就四项,每项是一句人话:

扣分项	人话	管什么
效率项	常用的货放得远,扣分	取货快
稳定项	重货放得高,扣分	安全
能耗项	货重×举升高度大,扣分	省电
预留项 δ	冷门货占了黄金位,扣分	给未来留后路

第四项 δ 是一个**仍在被反证约束的研究项**。它的设计动机像饭店留座:**门口的好位子要留给常客——**

一箱一年取一次的货,凭什么占离门口三步的格子?没有这一项,打分器会退化成

"哪近放哪"的倾向。早期实验只证明: $\delta=0$ 时**占用仓位集合**与 near 一致,不证明每件货的 gid 映射或算法严格等价;生命周期净收益后来被 drain 复算否定(实验发现 F2/F21)。

另外,sim 会离线生成一张**"场景→权重/可行性"表**。0712 fail-first 复核后,18 格中有 6 个高密度 online 格没有零失败候选,必须显示 **INFEASIBLE**; 120/skew online 的 2-seed 赢家也不稳定。

这张表目前是 sim 设计与风险清单, **PLC 在线查表闭环尚未接通**, 不能把 "有表" 说成 "已交付自适应" 。

第二层:AWRA-LS(夜里进场的"整理师")

如果一整批货的名单提前知道(比如一次到 120 箱),就不用一箱一箱贪心了。三步:

1. **Rank 排队**:重要的先挑位——频次高的、重的、大的排前面(就像登机先让头等舱);
2. **Assign 逐个放**:按队伍顺序,每箱还是用同一个打分器选格子;
3. **Local Search 微调**:放完后两两"换换看"——换了总分能降就换,降不动为止。

它的意义是给出**"信息全知情况下能做到多好"的上界**:在线打分器离它差 22.7%

(H 口径),这个差距就是"不知道未来"的代价——报告里这条 gap 分析是主线故事。

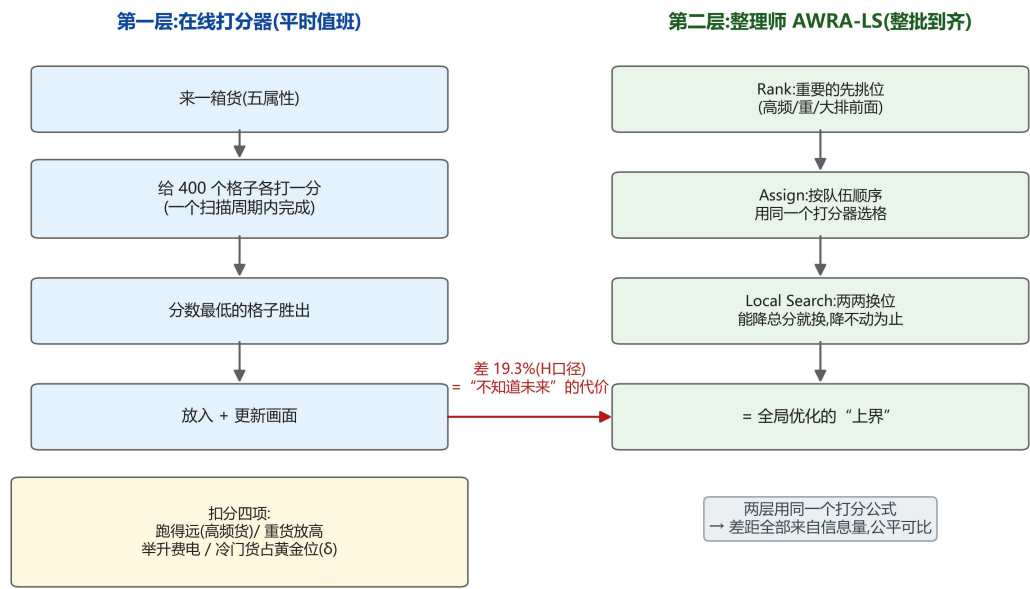


图 E2 双层算法一图流:左=平时值班的打分器,右=整批到齐的整理师(示意图)

成绩单 (sim H 设计口径=梯形加减速+自适应权重; 5-seed 回归锚; 来源 headline.py)

策略	期望取货时间	一句话
随机放	22.53 秒	无脑基线
顺序放(题面基线)	22.12 秒	从(0,0)挨个塞
就近塞	11.92 秒	快,但重货乱放
在线评分(我们)	10.87 秒	实时可用
AWRA-LS(我们)	8.40 秒	批量上界

配套指标:比题面基线**取货快 62.0%、提升耗能省 67.3%**,最重的 20% 货平均放在

0.57 层(几乎全贴地),**安全违规 0 次**(任何实验里都必须是 0,这是底线不是指标)。

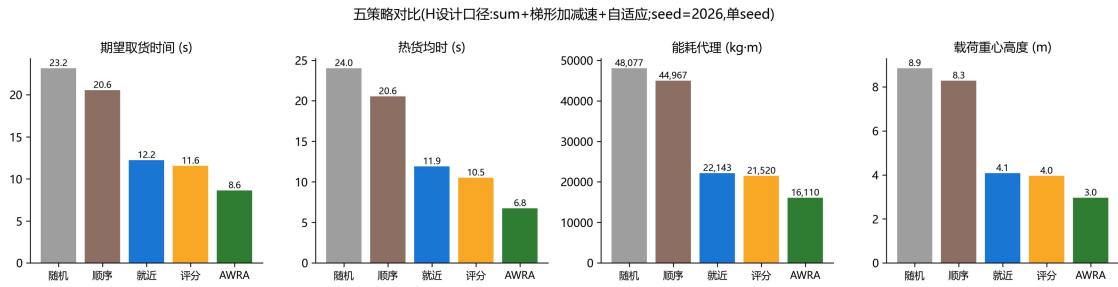


图 1 五策略主指标对比(真实数据图,复现:python sim/warehouse_sim.py)

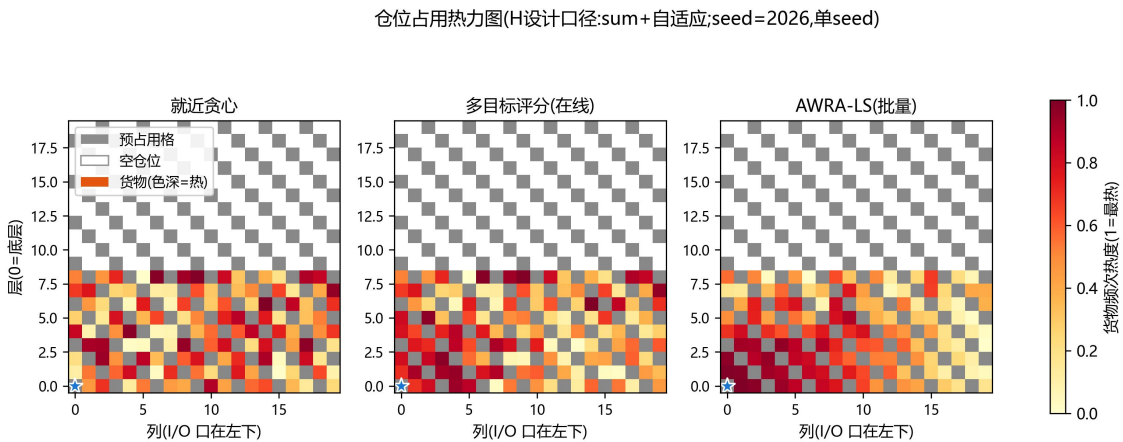


图 2 布局热力图:优化后热门货(深色)自动聚到左下角 I/O 口附近——算法在“看得见”地工作(真实数据图)

3. 我们对比过哪些算法(不是只做了一个就交差)

对比对象	它是什么	结果
随机 / 顺序 / 就近	三个基线(顺序=题面写法)	全被显著超过(上表)
NSGA-II	学院派多目标进化算法(论文里最常见的“高级货”)	我们手写了一个来打擂台:它的解没能压过 AWRA-LS;它的价值是画出“取货时间 vs 能耗”的取舍前沿图(跑死→救活的全过程见 §7.1)
下界与可达候选	下界给不可能突破的方向, 可达候选证明至少有人做到	当前静态匀速 sum 口径: $5.543 \leq 5.943 \leq 6.53$; AWRA 距下界 17.8%, 距已知可达候选 9.9%。5.943 未被证明是全局最优, 不能称“距天花板 9.9%”

强化学习(RL/DRL)	时髦的 AI 路线	自己实跑了轻量版(见§7.7): 1200 回合后 9.66s, 较在线评分 9.09s 差 6.3%, 与标定方向余弦 0.36; 它没有支持δ卖点。 深度版只作公开文献对照
--------------	-----------	--

还有三个"防杠"实验:换货物分布、仓库塞到 240/267 满(90%,行业设计上限)、重货翻倍——我们的方法退化最平缓

(鲁棒性,F12);权重、物理参数上下扰动一大圈,结论不变(敏感性,F15)。

一句话:不是选了个算法,是选了个算法并证明了为什么是它。

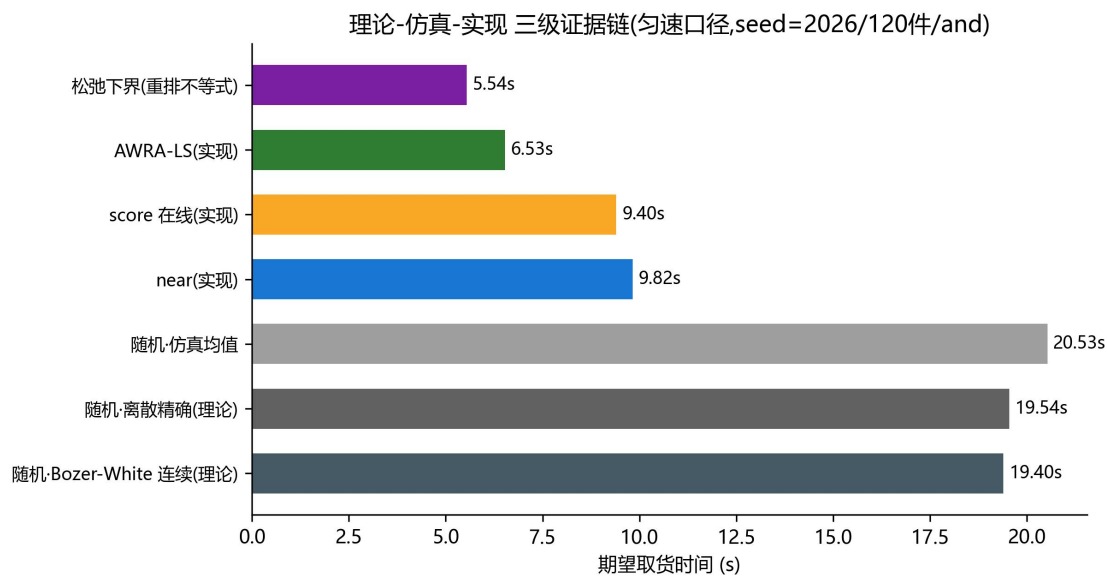


图 9 夹逼阶梯图：数学下界≤NSGA 已知可达候选≤AWRA≤各基线；9.9%是距候选点，不是距理论天花板

4. 成熟的工业产品一般用什么算法?(以及我们和它们的关系)

真实仓库软件(WMS)和自动化仓库厂商,几十年来主流是这一族——没有魔法,全是规则+优化:

工业界做法	大白话	对应我们的哪块
ABC 分类	按出货频率把货分三六九等,A类放黄金区	我们评分的"效率项"是它的连续版(不用分档,直接算)

COI 指数 (1963 年经典,至今在用)	体积÷频次,值小的放近处	同上,我们的打分类是这类规则的多目标推广
货位优化模块 (商业 WMS 的付费功能)	离线批量算一份"搬家建议单"	=我们的 AWRA-LS
夜间重排 / re-warehousing	白天乱了,夜里闲时把货挪回该在的位置	=我们的周期重排闭环(T10),还给它算了"值不值得搬"的经济账
切比雪夫行程模型 + 双命令周期	堆垛机双轴同时动;一趟顺路"存一箱+取一箱"省空跑	我们的运动模型与作业流,教科书同款
前沿试点:RL/DRL	论文和少数试点	我们自跑了轻量版(\$7.7)+深度版文献对照(\$3)

结论(这也是答辩定位):我们没有发明外星科技,而是把工业界**真实在用**的这一族方法, 做成了一个"可解释、可复现、有理论验证、真的能跑在 PLC 上"的完整小系统。

2025 年同类赛道一等奖的评委评语原话是"准确、可运行、清晰、合理"——没有一个字是"炫"。

5. 为什么算法要装进 PLC?难在哪?

PLC 是车间里的工业电脑,特点是**极其守时**:每 10 毫秒必须把所有活干完一轮, 超时就被"看门狗"(超时自动重启的保护机制)咬——所以不能让一个要算几秒的优化算法 直接塞进去,那等于在收银台前趴下算奥数。

我们的解法叫**扫描周期分片**:把大作业切成小片,每个 10 毫秒周期只处理几箱货/几百个 换位判断,像"每天只写两页作业,一周写完"——算法照常收敛,PLC 全程不卡。

这是"AI 算法落地 PLC"的核心工程手段,也是 AC500 实现 30 分里我们最硬的一张牌。

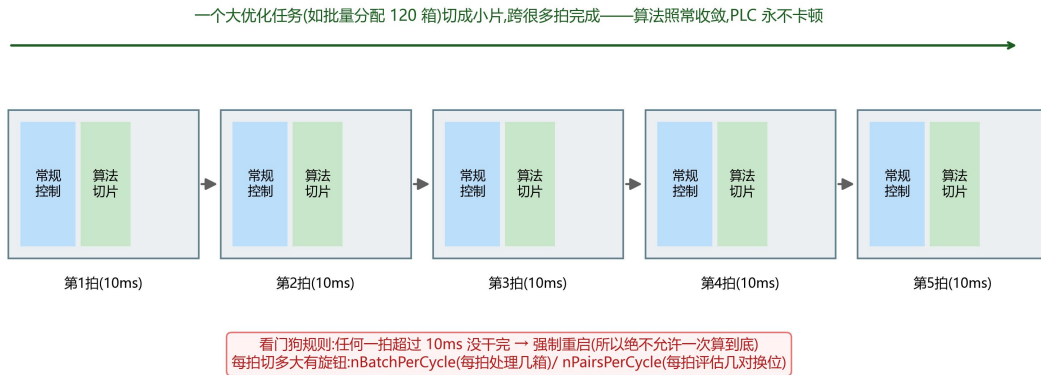


图 E4 扫描周期分片:大任务切成小片塞进 10ms 心跳,看门狗全程不咬(示意图)

另一张牌:**同一套算法养了对双胞胎**——Python 版(在电脑上做实验、出图表)和

ST 版(在 PLC 里干活)。两兄弟定期"对暗号":自动生成几十组测试数据,两边各算一遍,

答案必须按已锁向量一致。当前可引用证据是 **59/0 静态匀速黄金向量+核心 ST 用例在 AB PC 仿真通过**; 它不覆盖 H、自适应、lex/speed 或实体目标机。工作区后续虽出现 75 项源码, 但 T63-T75 仍含占位且未 AB 复测, 不能把 75 当新证据。报告把这称为**有限一致性护栏**, 不称完整数字孪生闭环。

6. 到今天为止,我们都做了哪些努力(资产盘点)

按时间线讲这个项目的故事:

4. **搭骨架**(7/03):读题→查文献→定双层架构→Python 仿真器+PLC 代码骨架一天成型;
5. **六个升级**(7/04):匀速改成真实的加减速模型;补上"取货"半边(题面要求存取,

大多数队只做存);理论下界+NSGA-II 对照;能耗换算成"一年省多少度电"

(sim 净机械功情景年省 266.5 度、48.9%, 假设全部写明; 0712 口径);自适应权重表;鲁棒性压力测试;

6. **控制器侧有限验证**(7/04-12):AB PC 仿真中核心 ST 逻辑+静态匀速黄金向量 59/0 通过;这不是实体 AC500 性能实测, 也不覆盖 H、自适应或 lex/speed 闭环;

7. **情报战**(7/05):挖出往届获奖率(4.9%~10.6%)、评委评语原话(=我们报告的验收标准)、官方提交的加密邮件规矩(差点所有人都不知道)、给赛道站台的 ABB 高管原话(报告引言直接呼应);

8. **方法复核**(7/05-12):报告/PPT 生成器、敏感性分析与可视化持续校准; 0712 补上 fail-first、共同到达流和 lifecycle drain, 主动撤回不成立的结论。后续仍有 PLC 路由、可视化长跑和真机验证待做。

现在手里的资产:18 条带编号的实验发现(每条附复现命令)、16 张报告级图表、

20+ 份数据表、59 个核心 ST 静态匀速黄金向量用例 (AB PC 仿真 59/0)、报告/PPT 草稿各一份(数字自动从数据文件注入,

改数据重跑一下就更新)、一套零基础可视化搭建操作卡。

7. 深度尝试实录:我们踩过的坑与挖过的井(这章是项目的"可信度来源")

上面几章都是打磨好的结论。这一章讲结论**是怎么来的**——试了什么、怎么撞的墙、

怎么解剖、结论怎么反转。评委评语奖励"准确、可运行",而准确从来不是没犯错,

是每个错都被解剖过。每个故事末尾附一句"值不值"判词。

7.1 NSGA-II:跑死、救活、再证明"不必救得更好"

试了什么:手写了一个零依赖的 NSGA-II(学院派多目标进化算法)来打擂台,60 个体×200 代。

撞墙:纯随机起步的版本跑满 200 代,最好也只到 6.67~7.76 秒一带——被我们 AWRA-LS 的单点(5.39 秒)整体碾压,进化基本白跑。

解剖:它的"杂交"环节出了结构性问题——两个好解交配出的孩子,大概率两箱货抢同一个格子,随机修复在拆散冲突时恰好把父母最值钱的基因(重货在低、热货在近)砸烂。孩子几乎永远不如

爹妈,进化等于原地踏步。

救活:把我们自己的 AWRA 解族“播种”进初始种群，前沿立刻推进；随后 200 代在该实现/预算下零改进。这证明播种重要，不证明全局最优。

结局:得到一个**已证可达候选点**；数学下界提供另一端参照。二者夹住当前已知空间，但更强优化器仍可能在区间内改进。

判词:已投入的一天买到了锚点+前沿图+"对照过学术标准"的可信度,很值;

再调一天,只会在夹缝里挪,还可能把我们的 gap 数字变难看而真相不变——不值。(F6/F9)

(本节 4.72/4.96/5.39 为 and 历史快照；当前 sum 链为 $5.543 \leq 5.943 \leq 6.53$ ，距候选 9.9%、距下界 17.8%，见实验发现 F18。)

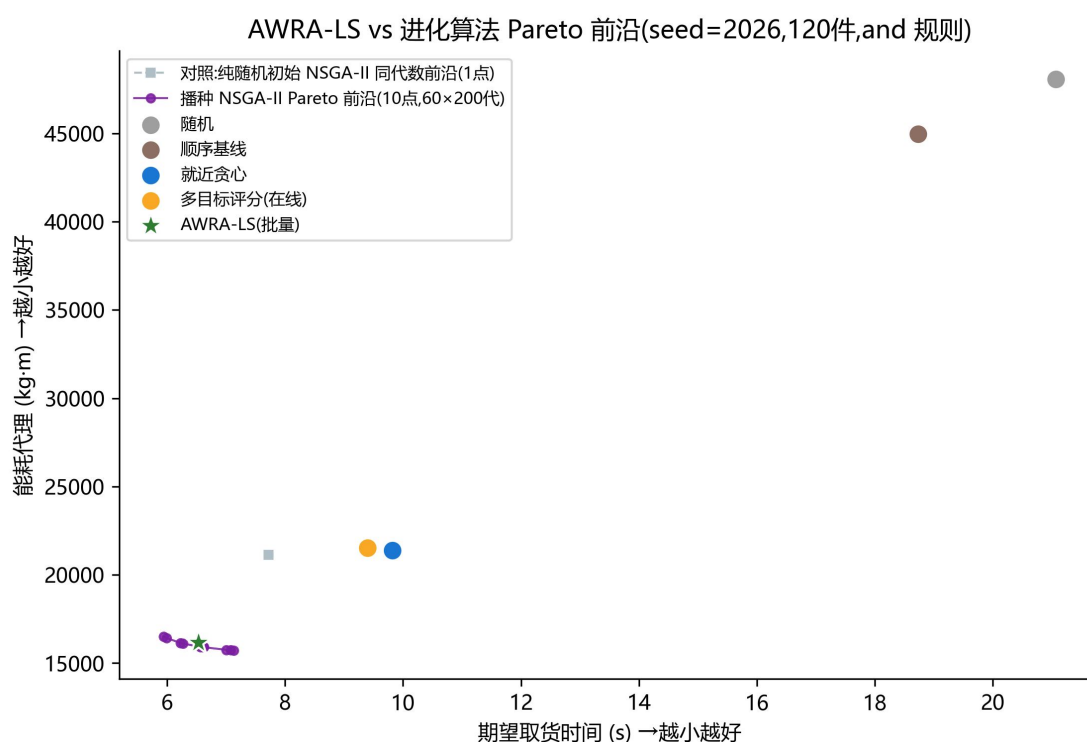


图 6 NSGA-II 打播结果:救活后的前沿(点群)与 AWRA-LS(星标)——前沿由播种解撑起(真实数据图)

7.2 δ 的一整条证据链:一个参数是怎么“挣得”存在资格的

δ 最初被解释为“给未来热门货留黄金位”。早期静态扫描看到 0.10~0.30 平台； $\delta=0$ 只在**占用仓位集合**上与 near 一致，不证明 gid 映射或严格算法等价。0712 方法复核改变了结论：

fail-first 下 200/240 件的六个 online 格全部 INFEASIBLE; 加入 drain 后, 120 件生命周期中 $\delta=0.15$ 总时间显著更差, 240 件只见失败数减少且两组仍大量失败。

判词: δ 现在是“有设计动机、但当前生命周期收益被证伪”的研究项, 不是招牌卖点。保留这条反转, 比掩盖负结果更可信。

7.3 重排翻车记: 净亏 497% 的实验怎么变成加分项

试了什么: 仓库跑久了布局会乱(后到的货被迫放次优位), 我们做“夜间自动整理”。

撞墙: 第一版逻辑是“只要挪了能改善评分就挪”——结果一夜搬 660 次, 五夜烧掉 63105 秒

行车时间, 换来 3.2% 的质量改善, **净亏 497%**。要是不细看, 差点得出“重排有害”的错结论。

解剖: 两个坑叠加。①搬家不要钱吗? 没算搬运本身的行车成本; ②作业流模型用错了——

“一次性入库流”里每箱货最多被取一次, 重排收益天然封顶; 题面说的“访问频次”本义是循环取用, 必须换成循环访问口径再评。

结局: 改成经济决策(预期收益 $> 1.2 \times$ 搬运成本才动手, 每夜限额 20 次) + 正确口径,

重排的真实身份水落石出: **它不是提效机制, 是自愈机制**——平时别乱动, 布局被扰动后

把系统拉回最优。压力测试(F12)从另一侧验证了这个定位: 在线策略决定“平时多好”,

重排决定“坏了之后多快好回来”。

判词: 一个 497% 的翻车, 解剖后变成报告“工程取舍”章最有说服力的一页——

负结果如实写, 比十页成功更显成熟。极值。(F11)

! 重排闭环:在线智能 × 重排策略(不排/每夜/漂移触发)对照(重排算子=FB_LocalSwapImprove,PLC 零新增;触发式虚线线与每夜实线重合=本负载下每

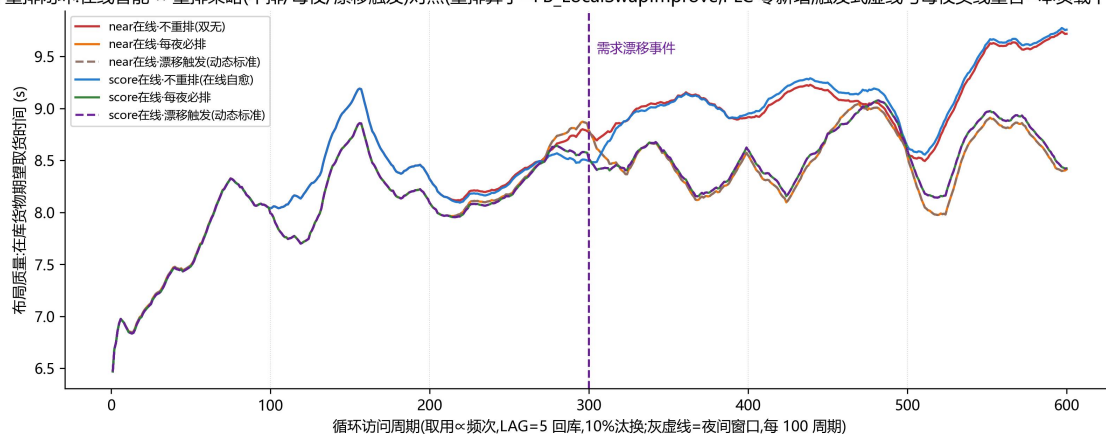


图 11 重排闭环:布局质量随时间漂移(升=变差),夜间自愈把它拉回来(真实数据图)

7.4 换尺子事件:匀速模型骗人最狠的地方,恰好是你最得意的地方

许多简化模型默认“堆垛机匀速跑”。我们换成更工程化的梯形加减速 sim 后发现一个陷阱:

匀速尺子对**随机布局**只低估 11%,对**就近贪心**低估 32.5%,对**我们优化后的布局**低估 **43.5%**

——因为优化把货全聚到近处,而短途恰恰是加减速占大头、匀速假设最失真的区间(F7)。

翻译成成人话:**布局做得越好,旧尺子量出来的成绩越虚**。这就是为什么主口径必须换尺子,

哪怕换完数字变“难看”(当时 7.42 vs 5.33; 当前 8.40 vs 6.44)——我们主动选择更保守、假设更完整的 sim 口径;仍不把它称为实体设备真值。

判词:一天工作换来“我们连测量工具都审过”的姿态,评委里坐着 ABB 工程师,这一页是内行话。

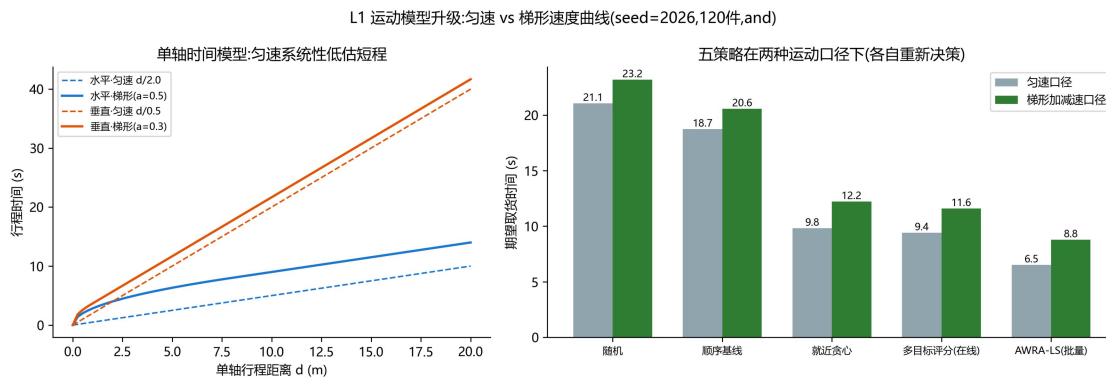


图 7 两把尺子量同一批布局:越优化的布局(右侧策略),匀速尺子低估得越狠(真实数据图)

7.5 LTIME=0 之谜:测不出来时,先怀疑表,再怀疑世界

撞墙:想实测"算法切片占用 PLC 每周期多少微秒",探针放进去被调用了五万多次,

读数**全是 0**。

解剖:探针没坏(采样计数在涨),坏的是"表"——最可能的根因是 PLC 的时间函数在

一个扫描周期内返回缓存值(整个周期读到的是同一个时刻),周期内无论怎么测都是 0。

这个推断顺手枪毙了一个看似聪明的备选方案("累计一千次采样摊薄分辨率"——一千个 0

加起来还是 0),少走一轮注定失败的弯路。

结局:LTIME/任务监视/SysTimeGetNs 在 AB PC 仿真中都拿不到周期内非零微秒值,最终只保留"完成所需任务周期数"。周期数是历史 AB PC 仿真观察,乘 10ms 只是标称换算,不是实体 AC500 实测。

判词:值——"测量手段本身要被怀疑"是工程素养的硬证据,而且失败原因分析防止了

后续瞎忙。(README_PLC §4b)

7.6 三次"人祸"防御战:贴错窗、误诊、8.7% 伪值

深井不全在算法里,有三次是防人的:

- **贴错窗事故:**执行助手(Codex)自己坦白可能把代码贴错了窗口又恢复了——恢复后没人重新验证过,那批通过的用例就只是"预期"不是"事实"。我们强制重验,当前只引用有截图/读回依据的 59/0;

文件时间戳闭合证据链(19:08 编译零错误 → 19:11 测试全过)。

- **误诊否决:**Codex 声称"程序直接挂任务会报 not defined"并想把这写进报告——复核后结论不成立(它自己的绕法碰巧等效),这条被明令禁止进任何文档。执行者的结论也要审。
- **gap 两次纠错:**先发现 5.39/4.96 舍入后写 " $\leq 8.7\%$ " 不成立,原始值是 8.77%; 0712 又进一步纠正语义——4.96/5.943 只是已知可达候选,不是全局最优。当前 sum 口径应写"距候选 9.9%、距松弛下界 17.8%",不写"距最优"。

判词:三次都值——比赛输在这种地方比输在算法上冤十倍。

7.7 RL 亲测记:AI 自学 1200 回合,学会了"就近",学不会"留位"

试了什么:不满足于只引文献,我们让一个轻量强化学习(线性策略梯度)从零自学放货:

不给任何先验,只给"最终取货快=奖励",练 1200 回合(每回合 120 箱)。

0712 重跑结果:从随机起点 19.19s 练到 9.66s (改善约 49.6%, 违规 0); 在线评分为 9.09s, RL 仍差 6.3%, AWRA 批量参考为 6.19s, RL 差约 56%。学得方向与标定方向余弦 0.36。

最有意思的发现:它自始至终没学出 δ 预留逻辑(学到的权重方向与我们标定的方向

余弦只有 0.33)。原因很本质:"把好位子留给未来的常客"是前瞻性决策,功劳要到很多步

之后才兑现,RL 的终局奖励很难把账算到当初那一步"忍住不占"上(行话叫信用分配难题)。

判词:极值——一个下午的实验买到三样:①工程取舍从"引文献"升级为"自己跑过";

② δ 证据链多了反向一环:**预留逻辑是设计出来的,不是机器白捡的**;③评委问"为什么不用

强化学习"时,我们有自己的学习曲线图。(F16,fig15)

7.8 判断"值不值得再深挖"的统一框架(所有取舍共用)

每口井挖不挖、挖多深,都过同三问:

9. **边界由什么决定?**(数学下界、已知可达点、物理规律、评分目标); 下界和候选点只给参照,不证明优化空间被封死。

10. **再投一天,评委能看到的增量是什么?**(看不到的努力=没有努力;这也是 40 天

转向报告/录屏/PPT 的全部理由)

11. **失败能不能变成叙事资产?**(497% 翻车、LTIME=0、NSGA 冻结——全都能;

所以敢试,试错不亏)

8. "口径"是什么?为什么同一个东西有两个数?

当前你会看到“8.40 秒(sim H 设计口径, 5-seed 回归锚)”和“6.44 秒(匀速+静态 sim 对照)”并存——不是错,是**两把尺子**:

- **H 设计口径**:sim 里开梯形加减速+场景权重; 不是 PLC H 闭环, ± 0.48 是 sample SD 而非 CI;
- **对照口径**:匀速 + 固定权重——和学术文献一个量法,方便比对。

其实"两把尺子"只是冰山一角:全项目一共 **9 对尺子**——运动模型(加减速/匀速)、权重(自适应/固定)、行程度量(时间/长度)、程数(双程/单程)、预占用解读(三种)、作业流(循环/一次性)、轴联动(同动/顺序)、能耗回收(保守/再生展望)、负载实测(仿真/真机)。每对都有主口径和对照口径、都有切换开关,报告附录里一页"口径矩阵"看全。

可保留的主张是: 120 件 H 回归锚与三档行程降幅方向一致; 高密度 online 已出现 INFEASIBLE,

生命周期 δ 收益也被 drain 复算否定, 所以不能再泛化成“所有尺子只改数字、不翻结论”。

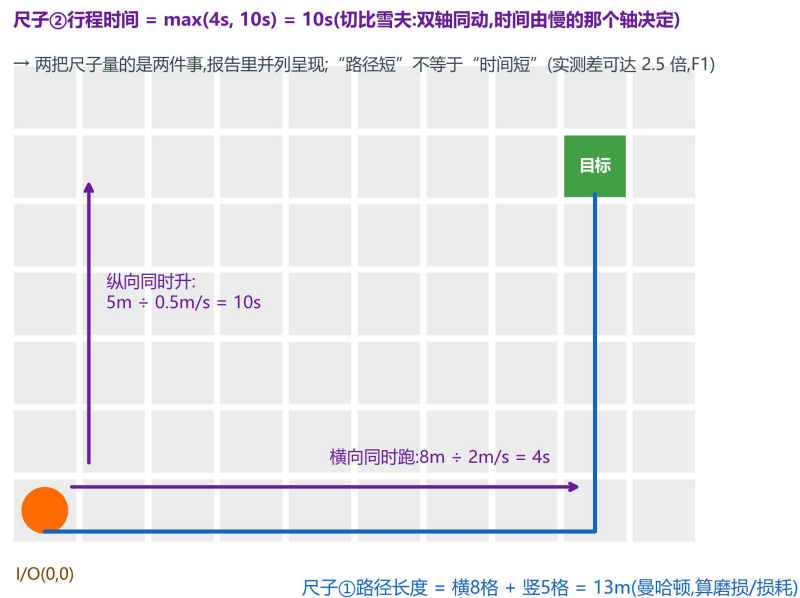


图 E3 两把尺子:同一趟行程,"路径长度"按横+竖算(磨损),"行程时间"由慢的那根轴决定(耗时)(示意图)

规矩(全项目铁律):任何数字必须说明用的哪把尺子;头条数字只认 [sim/headline.py](#)

跑出来的;文档里不许手抄数字。夹逼必须同时写“距已知可达候选 9.9%”与“距松弛下界 17.8%”，禁止把候选点改称全局最优。

口径还有个升级版:把“模型有多真实”本身也分了三档并排跑——见 §13「三档拟真度」。

9. 0712 之后还缺什么?你需要做什么?

大局判断：sim 方法修复、报告/PPT 生成器和用户文档已进入再生 QA，但技术并未全完成。PLC 的 TOB/CB 路由、当前 75 项中的占位断言、AB 新一轮复测、可视化在线长跑和实体目标机性能仍是硬缺口。

时间	干什么(一句话版)
1	Claude 复核/完成 ST 真实断言；禁止把 T63-T75 占位当通过
2	用户在场运行 AB 同步、编译、PRG_Test 与 400 件/watchdog 门，形成新截图/读回证据
3	完成三页可视化 GUI 洗、输入动作在线长跑与录屏
4	对当前 DOCX/PPTX、提交 manifest 与依赖清单做终审

你本人只有三类事：

- 12. **传令**:把固定那句话发给 Codex(信箱机制),它就去干活;
- 13. **AB 在场复测**：ST 由 Claude 收口后，你按操作卡跑真实断言并保存证据；
- 14. **学习**(每周一课,这份文档就是课 0):不为上台(初赛不答辩),为的是这个项目

从头到尾是**你的**——决赛阶段(8/26 后)如果要答辩,底子现在就攒着。

10. 常见疑问快答(FAQ)

- 为什么“违规=0”反复强调? 违规是资格线, 但必须与失败分列。有效头条与当前列出的 sim 行承重/容积复查为 0; 高密度 online 仍有 INFEASIBLE, 不能用“违规 0”掩盖放不进。
- 为什么随机数要"固定种子"? 让每个算法面对同一批货比赛才公平,也让评委在自己

电脑上跑出和报告一模一样的数——"可复现"是拿分点。

- 9.9%与 17.8%是什么意思? 9.9%是距一个已知可达候选点, 17.8%是距松弛下界; 前者不是全局最优, 后者也不说明可达, 因此只能作为内部夹逼参照。
- 为什么不把最时髦的 AI 全堆上去? 创新只占 10 分,而"准确、可运行、清晰"占大头;

文献里时髦方法的收益是个位数,代价是黑箱+要一年数据。我们把"为什么不用"写成了报告的一章——知道为什么不用,比什么都用更显专业。

- 拿不到真机 PLC 会吃亏吗? 当前 59/0 与任务周期数都来自 AB PC 仿真; 它们能证明有限逻辑/执行代理, 但不能替代实体 AC500 的最坏时延、I/O 与 watchdog 验证。报告必须把这项缺口明写。

11. 词汇表(见到就查)

词	人话
SLAP	仓位分配问题(这道题的学名,研究了 60 年)
堆垛机	立体仓库里那台"横着跑+竖着升"一体的取货机器
切比雪夫时间	双轴同时动时,行程时间=横竖两个方向里慢的那个
曼哈顿距离	横向距离+纵向距离(算路径长度/磨损用)
双命令周期	一趟顺路干两件事:送一箱进去+带一箱出来,省空跑
PLC	车间用的工业电脑,极守时、极皮实(我们用 ABB AC500)
ST 语言	写 PLC 程序的语言(长得像简化版 Pascal)
扫描周期	PLC 的心跳:每 10 毫秒把全部程序跑一遍
看门狗	超时保护:一个周期干不完活就强制重启——所以

	算法必须切片
FB(功能块)	PLC 程序里的"乐高积木",评委点名要看封装是否合理
AWRA-LS	我们的批量算法:排队(Rank)→逐个放(Assign)→换位微调(Local Search)
δ (预留项)	打分公式第四项:冷门货占黄金位要扣分("好位子留给常客")
NSGA-II	经典多目标进化算法,我们请来打擂台的对照组
DRL	深度强化学习,时髦 AI;本项目只做文献对照
H 口径 / 对照口径	两把 sim 尺子:H 设计口径=梯形加减速+自适应权重 / 对照口径=匀速+固定权重(文献可比)
数字孪生	目标是让 sim 与 PLC 按同一规则互证;当前只完成核心 ST+静态匀速黄金向量 59/0,不能称 H 全链闭环
seed(种子)	让"随机"每次都随机得一模一样,保证公平与可复现
AB	Automation Builder,ABB 官方编程软件(建工程/仿真/可视化都在里面)
WMS	仓库管理软件(工业界真实产品,\$4 讲了它们用什么算法)

12. 课 0 小测(答完对答案;全对才算这节课通过,错题=下一课重点)

15. 我们的两层算法各叫什么?各自用在什么场景?
16. δ 预留项是干什么的?把它设成 0 会发生什么?
17. 为什么“期望取货时间”同时有 8.40 秒和 6.44 秒两个数?它们各是什么 sim 口径?
18. 优化算法进 PLC 为什么必须"切片"?不切会怎样?
19. 为什么不能说“离理论天花板 9.9%”? 9.9%与 17.8%分别比较了什么?

(参考答案位置:①\$2 ②\$2/\$7.2 ③\$8 ④\$5 ⑤\$3/\$7.1。

答案发给 Claude,它会批改并把错题排进下一课——这是"课后小测"机制的第一次运行,

见 docs/协作方法_未知项工作法.md。)

下一课预告(每周一课,跟排期走):课 1「打分公式与 δ 的故事」(配 fig3/fig14 讲权重),

课 2「PLC 与扫描周期分片」(配 AB 画面实操),课 3「口径与复现:怎么向别人证明数字是真的」。

有任何一节看不懂,直接问,我重写到你懂为止。

13. 三档拟真度:我们怎么给模型定"真实等级"(0705 新增)

13.1 为什么要分档

你打游戏见过难度选择:简单/普通/困难。做仿真模型也有同一个问题——**你可以把世界简化到什么程度?** 简化多了,数字好看但没人信;什么都模拟,做不完也讲不清。

我们的答案(你拍板的):不选一个,三档都做,并排放桌上。

档位	名字	里面是什么	类比
档 1	数据模型	匀速、固定权重、货一到就处理、装卸瞬间完成、频次全知	物理题里"光滑水平面上的小球"
档 2	工程主口径(H)	堆垛机真实加减速 + 权重按场景查表	加了摩擦力的物理题——头条数字用这档
档 3	工业场景	档 2 再叠四层现实:订单像真仓库一样忽多忽少地来(泊松到达、有高峰低谷)、每次存取要花装卸时间(货叉伸缩)、系统对货物热度的了解是估计值会有误差、设备偶尔坏一下(修 10 分钟)	真实车间的一天

13.2 设计时的三个关键决定(这一节是"怎么设计"的核心)

决定一:三档必须"同一条流水线,只切开关"。三档跑的是同一批货、同一个作业流程、同一个随机种子,唯一区别是拟真开关开几个。这就是控制变量法——否则档与档之间数字没有可比性,阶梯就是假的。(实现上档 1/档 2 直接复用既有混合流代码,连数字都和

以前的实验逐位一致,这是自证没改坏。)

决定二:档 3 只做 sim 验证,不动头条。 头条数字(8.40 秒的 5-seed 回归锚)来自档 2 设计口径。

档 3 的任务是回答一个问题:"把现实全叠上去,你们的结论还站得住吗?"它是质检环节,不是新的宣传口径。这样设计还有个工程好处:档 3 随便加东西,都不会牵动全项目的数字锁和回归测试。

决定三:档 3 的每个参数都要"有出处"或"明说是假设"。 这是最花功夫的部分,

我们为此做了一整轮调研(见 13.3)。查得到的(装卸时间由厂商公开的货叉速度推导,还扫了 10/15/20 秒三个点证明结论不挑数),按出处写;查不到的(到达强度、故障频率这类没有公开统计的),就在文档里明写"这是情景假设,不是标定值"——**诚实声明比假装精确更值钱**。

13.3 为了"真实"我们查了多少东西(你的原话:这是我们的一大优势)

- **7 组真实堆垛机的参数表**(Mecalux、MIAS、国产机型.....):发现我们的速度取值恰好对应"轻载单立柱堆垛机"这一档的公开规格,而且垂直速度还取了保守值;
- **ABB 自家的文档**:堆垛机专项手册、起重机固件手册(里面参数 23.16 写着:填 0 = 线性斜坡(就是我们的梯形!),填正数 = S 曲线)——我们的建模选择和 ABB 固件的默认配置是同构的;再生制动的官方案例(32%~54% 回收)正好把我们的假设值夹在中间;
- **1984 年的开山论文原文**:堆垛机"水平垂直同时动、时间取较慢那根轴"这个假设,就是我们从第一天用到现在的切比雪夫模型——原文句子和 DOI 都存档了;
- **一份"防伪清单"**:调研中发现有些网上流传的数字是营销夸张(比如"回收 60%"),已列入黑名单,写报告时绝不引用。

13.4 跑出来什么(以及怎么读)

优化算法比题面基线快多少(出库均时降幅):**档 1 里 49.9%,档 2 里 43.7%,档 3 里 44.7%**

(官方口径 0706;切换前的首跑为 60/53/53,阶梯形状一致)。

读法:①越真实,数字越"难看"——匀速的档 1 把我们的优势夸大了约 6 个百分点,

我们主动不用它当头条;②从档 2 到档 3 优势几乎不掉——加上装卸、高峰、误差、故障,

行程排序保持、承重/容积复查为 0; ③档 3 还能看排队效应。共同到达流重算的单 seed 结果为:

P50 seq/near/awra=135/77/71s, P95=918/651/667s。AWRA 的 P50 最低, 但 P95 最低是 near;

这说明要逐指标报告, 不能用一个分位数包装“全面最优”。

一句话总结(也是答辩口径):**评委不会因为你说"我的模型很真实"就信你,但会因为

"每加一层真实,结论都还站得住"而信你。 ** 拟真度是用来做压力测试的,不是用来堆的。

(实验记录:实验发现 F17;图:fig16;参数出处:canonical G 节;调研战利品清单:

docs/L7_拟真度阶梯设计.md。复现:[python sim/realism.py](#),两分钟。)